

PLAN DE PRUEBAS

EXORDIUM: DATA ARCHITECTURE PLATFORM

SECCIÓN 1: INTRODUCCIÓN

1.1 Propósito del Plan de Evaluación

El presente documento establece el plan de evaluación para el proyecto de grado "Exordium | Data Architecture Platform". Este plan define las métricas, criterios, instrumentos y actividades necesarias para validar la calidad del software desarrollado, conforme a los atributos seleccionados del modelo ISO/IEC 25010:2023. El objetivo es proporcionar una guía estructurada y metodológica para la ejecución de pruebas durante el ciclo de vida del desarrollo del proyecto. *El plan se diseña bajo un enfoque iterativo, donde cada funcionalidad implementada es objeto de evaluación antes de su integración final al producto, asegurando una gestión proactiva de la calidad desde las fases tempranas del desarrollo.* La documentación sigue la estructura recomendada por el Taller de Planificación de Evaluación del Proyecto de Grado de la Universidad ICESI y se adhiere a los principios del estándar IEEE 829 para la documentación de pruebas de software.

1.2 Breve Descripción del Proyecto de Grado

Exordium es una plataforma web diseñada para automatizar la implantación de Arquitectura de Datos en organizaciones colombianas, basándose en el marco de referencia DAMA-DMBOK Capítulo 4. El sistema utiliza Inteligencia Artificial de Generación Ampliada por Recuperación (RAG) para asistir a los arquitectos de datos en la generación de 14 artefactos clave distribuidos en cuatro módulos funcionales: AS-IS (diagrama ER, inventario de datos, DFD, matriz RACI), TO-BE (ER objetivo, modelo lógico, glosario, DFD), Brechas (CRUD comparativo, reporte de brechas, reglas de negocio) y Roadmap (Diagrama de Gantt, Estándares, KPIs). La arquitectura tecnológica del proyecto se apoya en un stack validado que incluye Next.js 15 para el frontend, FastAPI para el backend, LangChain para la orquestación de flujos de IA, GPT-4o como modelo de lenguaje, PostgreSQL como base de datos relacional y Docker para la contenedorización e infraestructura.

1.3 Objetivos del Plan de Evaluación

Este plan de evaluación persigue los siguientes objetivos específicos:

Código	Objetivo
OE-01	Definir las métricas de calidad para evaluar tanto el proceso de integración técnica como el producto tangible (artefactos generados).
OE-02	Establecer criterios de aceptación factibles y medibles, alineados con las limitaciones y recursos disponibles en el contexto académico del proyecto de grado.

Código	Objetivo
OE-03	Seleccionar y documentar instrumentos de evaluación (herramientas y procedimientos) compatibles con el stack tecnológico y la metodología de desarrollo ágil-iterativa.

1.4 Justificación de la Evaluación del Proceso y del Producto

La evaluación de Exordium contempla de forma explícita y conjunta dos elementos centrales: el proceso y el producto. La justificación para esta doble perspectiva se detalla a continuación:

Evaluación del Producto: El núcleo técnico del proyecto reside en la orquestación del pipeline RAG, que integra componentes dispares como LangChain, FastAPI y Next.js. Validar la estabilidad y correcta comunicación entre estos elementos es una condición *“sine qua non”* para demostrar la viabilidad técnica del enfoque propuesto. En una primera versión mínima viable (MVP) de un proyecto de investigación aplicada, la demostración de que la arquitectura integrada funciona correctamente tiene prioridad sobre optimizaciones de rendimiento o escalabilidad. Por lo tanto, la evaluación del proceso se enfoca en la **Mantenibilidad y la Adecuación Funcional del propio flujo de trabajo.**

Los 14 artefactos de arquitectura de datos generados por la plataforma constituyen los entregables tangibles y el valor principal para el usuario final. La evaluación del producto se centra primordialmente en la correctitud estructural, es decir, si los artefactos producidos cumplen con formatos y esquemas predefinidos (ej. JSON válido, DOT para diagramas ER, CSV con encabezados definidos). Esta aproximación responde directamente a la directriz del tutor de que el proyecto abarca la fase de integración de LangChain, pero no la personalización o entrenamiento de modelos de IA. En consecuencia, la evaluación de la calidad semántica del contenido generado por el modelo seleccionado queda fuera del alcance de esta versión, concentrándose en la validez técnica de la salida; ***es decir el equipo no tiene a su cargo ni el diseño, desarrollo, implementación, del modelo, sino que trabajará sobre los modelos suministrados por el tutor del trabajo de grado, o en su defecto, sobre los modelos llms comerciales o de uso libre.***

Esta estrategia de evaluación permite cubrir los aspectos principales del proyecto dentro de las restricciones impuestas, garantizando que la sustentación académica pueda presentar evidencia sólida tanto de la solidez de la arquitectura como de la utilidad funcional de los artefactos generados.

1.5 Definiciones, Acrónimos y Abreviaturas

Término	Definición
DAMA-DMBOK	Data Management Body of Knowledge, marco de conocimiento para la gestión de datos.
E2E	End-to-End, pruebas que validan flujos completos de usuario desde el punto de vista del cliente.
ISO/IEC 25010	Estándar internacional que define los modelos de calidad para productos de software.
MVP	Minimum Viable Product, versión mínima funcional de un producto que valida un conjunto de hipótesis de mercado.
PDG	Proyecto de Grado, trabajo de culminación académica requerido para la obtención del título de ingeniero en la Universidad ICESI.
RAG	Retrieval-Augmented Generation, técnica de inteligencia artificial que combina recuperación de información de bases de conocimiento externas con la generación de texto por modelos de lenguaje.
Pydantic	Librería de Python utilizada para la validación de datos y la conversión de objetos mediante anotaciones de tipo.
pytest	Framework de pruebas de software para Python, conocido por su simplicidad y capacidad de extensión.
Jest	Framework de pruebas de JavaScript/TypeScript utilizado ampliamente en el ecosistema de React y Next.js.
Playwright	Herramienta de automatización de navegadores para realizar pruebas E2E en aplicaciones web modernas.

1.6 Documentos Relacionados

Código	Documento	Versión
DOC-01	Taller de Planificación de Evaluación del Proyecto de Grado - Universidad ICESI	2025
DOC-02	ISO/IEC 25010:2023 - Modelos de calidad de sistemas y software	2023
DOC-03	IEEE Std 829-1998 - Norma IEEE para la documentación de pruebas de software	1998

Código	Documento	Versión
DOC-04	The DAMA-DMBOK: Data Management Body of Knowledge	2ª ed., 2017

SECCIÓN 2: MARCO DE EVALUACIÓN

2.1 Conceptos Fundamentales de Evaluación en Ingeniería de Sistemas

La evaluación de software en el contexto de proyectos de ingeniería de sistemas es un proceso sistemático que busca determinar el grado en que un sistema o sus componentes cumplen con los requisitos especificados. Este proceso se fundamenta en tres dimensiones interrelacionadas que son cruciales para la comprensión y aplicación de este plan de pruebas:

Dimensión de Proceso: Esta dimensión se refiere a la evaluación de las actividades, métodos y herramientas empleadas en el ciclo de vida del desarrollo del software. Para Exordium, esto implica analizar la calidad de la arquitectura, la eficiencia de los flujos de trabajo de integración continua, y la efectividad de las prácticas de desarrollo ágil-iterativo. La evaluación del proceso se materializa en la verificación de la modularidad, la reusabilidad y la capacidad de prueba del código fuente, contribuyendo directamente a la mantenibilidad del sistema.

Dimensión de Producto: Esta dimensión se centra en la evaluación de los artefactos tangibles que genera el proceso de desarrollo. En el caso de Exordium, el producto principal es la plataforma web y los 14 artefactos de arquitectura de datos que esta genera. La evaluación del producto se enfoca en verificar que estos artefactos cumplan con los formatos y estructuras predefinidas, asegurando su utilidad para el usuario final y su compatibilidad con otras herramientas de diseño de software. Esta evaluación se realiza mediante pruebas de validación de esquema y pruebas de usabilidad exploratoria.

Dimensión de Calidad: Se basa en modelos estandarizados que definen características medibles para cuantificar la calidad de un producto de software. El marco de referencia principal para esta dimensión es el modelo ISO/IEC 25010:2023, que define un conjunto de características de calidad interna y externa. Este plan de pruebas adopta dicho modelo y selecciona tres atributos de calidad prioritarios que son medibles y relevantes dentro del alcance y las limitaciones del proyecto académico. Estos atributos son: Adecuación Funcional, Capacidad de Interacción y Mantenibilidad.

2.2 Relación entre Objetivos del Proyecto y Evaluación

Para garantizar que la evaluación sea relevante y efectiva, se establecerá una trazabilidad clara entre los objetivos del proyecto y los atributos de calidad que se van a medir. Esta relación asegura que cada métrica definida contribuya directamente a la validación de los logros del proyecto. La siguiente tabla detalla esta correspondencia:

Objetivo del Proyecto	Elemento a Evaluar	Atributo ISO 25010 Asociado	Métrica Asociada
Automatizar la generación de artefactos DAMA-DMBOK	Producto (Artefactos)	Adecuación Funcional	Porcentaje de artefactos generados con estructura válida según su esquema Pydantic.
Garantizar la estructura técnica válida de los artefactos generados	Producto (Artefactos)	Adecuación Funcional	Porcentaje de artefactos que cumplen con los formatos y esquemas técnicos definidos (JSON, DOT, CSV), sin evaluar la precisión semántica del contenido generado por el modelo de IA externo.
Proveer una interfaz navegable para usuarios	Producto (Interfaz de Usuario)	Capacidad de Interacción	Puntuación en un checklist de usabilidad básico, aplicado a una muestra pequeña de usuarios.
Garantizar la mantenibilidad del código base mediante la evaluación de las subcaracterísticas de analizabilidad, modificabilidad y capacidad de prueba, conforme al modelo de calidad de producto ISO/IEC 25010:2023.	Proceso (Desarrollo)	Analizabilidad, Modificabilidad, Capacidad de prueba	Cobertura de pruebas unitarias en los componentes del frontend y backend.
Integrar los componentes de forma estable y coherente	Proceso (Orquestación)	Adecuación Funcional	Tasa de éxito en las pruebas de integración entre los servicios de FastAPI y el pipeline de LangChain.

2.3 Atributos de Calidad ISO 25010 Seleccionados

Se han seleccionado tres atributos de calidad del modelo ISO/IEC 25010:2023, cuyo alcance y naturaleza son adecuados para un proyecto de grado en su fase inicial, considerando las limitaciones de recursos, tiempo y la directriz de no evaluar fiabilidad y seguridad avanzada. La selección se justifica por su pertinencia, factibilidad y capacidad para validar la viabilidad técnica del proyecto.

Atributo	Subcaracterísticas Enfocadas	Justificación del Alcance
Adecuación Funcional	Compleitud funcional, Corrección funcional	El atributo central del proyecto. Permite validar que los 14 artefactos se generan en estructuras válidas, lo cual es un requisito indispensable para que el producto tenga valor.
Capacidad de Interacción	Operabilidad, Reconocibilidad, Protección contra errores de usuario	Fundamental para la sustentación académica. Un sistema funcional pero inutilizable por el comité evaluador sería un fracaso. La evaluación se simplifica a una prueba exploratoria con una muestra académica pequeña, lo cual es realista y suficiente para un PDG.
Mantenibilidad	Compleitud funcional, Corrección funcional, Apropriación funcional	El atributo central del proyecto. Permite validar que los 14 artefactos se generan con: (a) todas las funciones requeridas según DAMA-DMBOK Cap. 4 (compleitud funcional), (b) esquemas estructurales válidos sin errores de formato ni densidad de errores crítica (corrección funcional), y (c) mensajes e información adecuada que guían al usuario durante el flujo de generación (apropiación funcional). La evaluación se limita a la estructura técnica de los artefactos, no al contenido semántico generado por el LLM.

Justificación de la Exclusión de Otros Atributos:

- **Eficiencia de Rendimiento:** Requiere la realización de pruebas de carga y estrés, las cuales dependen de escenarios de uso con alto volumen de datos y usuarios concurrentes. Estas condiciones no pueden ser replicadas en el contexto académico del proyecto de grado.
- **Fiabilidad:** Fue excluido deliberadamente por directriz del proyecto. La evaluación de la tolerancia a fallos y la recuperabilidad se aborda de forma básica en las pruebas de integración, pero no se mide como un atributo de calidad formal.
- **Seguridad:** Fue excluido deliberadamente por directriz del proyecto. Se realizarán validaciones básicas de seguridad para evitar vulnerabilidades graves, pero no se llevará a cabo una auditoría de seguridad exhaustiva.
- **Compatibilidad y Portabilidad:** Estos atributos se validan parcialmente de forma indirecta. La interoperabilidad entre componentes se verifica mediante pruebas de contrato OpenAPI, mientras que la portabilidad se asegura a través del uso de Docker Compose para un despliegue consistente en diferentes entornos.

SECCIÓN 3: MÉTRICAS Y CRITERIOS

Esta sección define el conjunto de métricas e indicadores cuantitativos y cualitativos que permitirán medir el grado de cumplimiento de los atributos de calidad seleccionados (Adecuación Funcional, Capacidad de Interacción y Mantenibilidad). Las métricas propuestas han sido diseñadas específicamente para ser ejecutables en un entorno académico controlado, sin dependencia de grandes volúmenes de datos, carga de usuarios concurrentes o despliegue en mercado real. Cada métrica incluye su fórmula de cálculo, el instrumento de recolección asociado y los criterios de aceptación que determinan el éxito de la evaluación.

3.1 Métricas para Adecuación Funcional

La Adecuación Funcional evalúa si el software proporciona las funciones necesarias para satisfacer las necesidades explícitas e implícitas del usuario. En el contexto de Exordium, esto se traduce en la capacidad del sistema para generar los 14 artefactos de arquitectura de datos con la estructura técnica correcta y completar los flujos de integración previstos

3.1.1 Porcentaje de Artefactos con Estructura Válida

Esta métrica mide la corrección funcional del producto, verificando que las salidas generadas por el pipeline de IA cumplan estrictamente con los esquemas de datos definidos.

Fórmula:

$$M_{AF-01} = \left(\frac{N_{validos}}{N_{total}} \right) \times 100$$

Donde:

- $N_{validos}$ es el número de artefactos que pasan la validación de esquema Pydantic sin errores.
- N_{total} es el número total de artefactos generados en la batería de pruebas.

Instrumento de Recolección: Scripts automatizados de prueba utilizando la librería `pytest` junto con modelos de validación `Pydantic`. Se ejecutarán contra un conjunto de 5 casos de prueba representativos (fixtures) que cubren diferentes escenarios de entrada para cada módulo (AS-IS, TO-BE, Brechas, Roadmap).

Criterio de Aceptación: El sistema debe alcanzar un valor igual o superior al **85%**. Un valor inferior indica que la orquestación del modelo de lenguaje o el post-procesamiento de datos no están garantizando la integridad estructural requerida para la utilidad del artefacto.

3.1.2 Tasa de Éxito en Flujos de Integración

Esta métrica evalúa la completitud funcional del proceso, midiendo la capacidad del sistema para ejecutar secuencias completas de generación de artefactos sin interrupciones críticas en la comunicación entre componentes (Frontend → Backend → LangChain → Base de Datos).

Fórmula:

$$M_{AF-02} = \left(\frac{E_{exitosos}}{E_{ejecutados}} \right) \times 100$$

Donde:

- $E_{exitosos}$ es el número de ejecuciones de flujo end-to-end que finalizan con una respuesta HTTP 200 y datos válidos.
- $E_{ejecutados}$ es el número total de intentos de ejecución del flujo en el entorno de pruebas.

Instrumento de Recolección: Pruebas de integración automatizadas utilizando `httpx` (TestClient de FastAPI) y `Playwright` para simular la interacción del usuario. Las pruebas mockearán las respuestas externas de la API de OpenAI para aislar fallos de red externos y centrarse en la lógica de integración interna.

Criterio de Aceptación: Se requiere una tasa de éxito igual o superior al **85%**. Esto asegura que la arquitectura es estable para fines de demostración académica, permitiendo márgenes mínimos para errores transitorios no críticos.

3.2 Métricas para Capacidad de Interacción (Usabilidad Automatizada)

La Capacidad de Interacción se refiere a la facilidad con la que un usuario puede operar y comprender el sistema. Para el contexto académico de Exordium, esta evaluación se realiza mediante herramientas automatizadas de accesibilidad y consistencia de interfaz, sin requerir intervención de usuarios finales. Esta aproximación permite obtener evidencia objetiva, reproducible y cuantificable conforme a las subcaracterísticas de operabilidad, reconocibilidad y accesibilidad definidas en ISO/IEC 25010:2023.

3.2.1 Índice de Accesibilidad y Consistencia de Interfaz

Esta métrica evalúa la operatividad, la reconocibilidad y la accesibilidad del sistema mediante validación automatizada de estándares técnicos (WCAG 2.1 AA), consistencia de componentes y ausencia de errores de interfaz

Fórmula:

$$M_{CI-01} = \left(\frac{Componentes_{validados}}{Componentes_{totales}} \right) \times 100$$

Donde:

- *Componentes_{validados}* es el número de componentes UI que cumplen con todos los criterios de accesibilidad (axe-core), consistencia visual (Playwright) y nomenclatura estándar (ESLint).
- *Componentes_{totales}* es el número total de componentes UI evaluados en la aplicación.

Instrumento de Recolección:

- **axe-core** integrado con **Playwright**: Validación automática de accesibilidad WCAG 2.1 AA (contraste de colores, etiquetas ARIA, navegación por teclado, focos visibles).
- **Lighthouse CI**: Puntuación automatizada de accesibilidad, mejores prácticas y performance.
- **Pruebas de regresión visual con Playwright**: Verificación de consistencia de diseño entre iteraciones mediante comparación de screenshots.
- **ESLint con reglas personalizadas**: Validación de nomenclatura en inglés y convenciones de componentes (PascalCase para componentes React).

3.3 Métricas para Mantenibilidad

La Mantenibilidad evalúa la facilidad con la que el software puede ser modificado para corregir defectos, mejorar el rendimiento o adaptarse a cambios en el entorno. Esta dimensión es crucial en un proyecto académico para demostrar rigor ingenieril y facilitar la continuidad del trabajo por futuros investigadores.

3.3.1 Cobertura de Pruebas Unitarias

Esta métrica mide el porcentaje del código fuente que es ejercitado por las pruebas automatizadas, sirviendo como indicador de la capacidad de prueba y la robustez ante regresiones.

Fórmula:

$$M_{M-01} = \left(\frac{L_{cubiertas}}{L_{ejecutables}} \right) \times 100$$

Donde:

- $L_{cubiertas}$ es el número de líneas de código ejecutadas durante la suite de pruebas.
- $L_{ejecutables}$ es el número total de líneas de código ejecutables en los módulos críticos (excluyendo configuraciones y comentarios).

Instrumento de Recolección: Herramientas de análisis estático integradas en el pipeline de CI/CD local: `pytest-cov` para el backend (Python/FastAPI) y `Jest --coverage` para el frontend (Next.js/TypeScript). Se generarán reportes HTML y XML consolidados.

Criterio de Aceptación: Se establece un objetivo de cobertura mínima del **80%** en los componentes de lógica de negocio y servicios de integración. No se exige cobertura del 100% para evitar la sobre-ingeniería en componentes puramente visuales o de configuración, manteniendo el esfuerzo en equilibrio con el alcance académico.

3.3.2 Cumplimiento de Estándares de Código

Esta métrica evalúa la analizabilidad del código fuente, verificando el cumplimiento de estándares de nomenclatura, formato y ausencia de código duplicado o sin usar, conforme a las directrices de revisión estática establecidas.

Fórmula:

$$M_{M-02} = \left(\frac{Lineas_{cumplen_estandar}}{Lineas_{totales}} \right) \times 100$$

Donde:

- *Lineas_{cumplen_estandar}* es el número de líneas de código que cumplen con todos los estándares definidos (nomenclatura en inglés, convenciones de mayúsculas para interfaces, ausencia de código duplicado, ausencia de código sin usar).
- *Lineas_{totales}* es el número total de líneas de código fuente del proyecto (excluyendo comentarios y líneas en blanco).

Instrumento de Recolección: Análisis estático automatizado con las siguientes herramientas:

- **Backend (Python):** `Pylint` con configuración estricta, `flake8` para verificación de estilo PEP8, `vulture` para detección de código sin usar.
- **Frontend (TypeScript/JavaScript):** `ESLint` con reglas personalizadas, `Prettier` para formato consistente.
- **General:** Verificación de nomenclatura en inglés (variables, funciones, clases, interfaces) y convenciones de mayúsculas para interfaces (PascalCase).

3.4 Matriz de Trazabilidad: Requisitos - Métricas - Pruebas

Para garantizar que todos los objetivos del proyecto estén cubiertos por el plan de evaluación, se presenta la siguiente matriz de trazabilidad. Esta tabla vincula los requisitos funcionales y no funcionales derivados de los objetivos del PDG con las métricas definidas anteriormente y los casos de prueba asociados.

ID Requisito	Descripción del Requisito	Atributo ISO 25010	Sub Característica	ID Métrica	Instrumento de Medición	Caso de Prueba Asociado
REQ-01	El sistema debe generar los 14 artefactos DAMA-DMBOK en formatos estructurados válidos.	Adecuación Funcional	Corrección funcional	M_AF-01	Validación Pydantic + pytest	CP-AF-01: Validación de esquema JSON para Diagrama ER
REQ-02	El sistema debe integrar correctamente el frontend, backend y el pipeline de IA sin fallos de comunicación.	Adecuación Funcional	Compleitud funcional	M_AF-02	Playwright + httpx TestClient	CP-AF-02: Flujo E2E de generación de matriz RACI

ID Requisito	Descripción del Requisito	Atributo ISO 25010	Sub Característica	ID Métrica	Instrumento de Medición	Caso de Prueba Asociado
REQ-03	La interfaz debe ser accesible, consistente y permitir navegación intuitiva sin barreras de usabilidad.	Capacidad de Interacción	Operabilidad, Reconocibilidad, Accesibilidad	M_CI-01	axe-core + Lighthouse CI + Playwright	CP-CI-01: Validación automatizada de accesibilidad WCAG 2.1 AA
REQ-04	El código debe ser modular, contar con pruebas automatizadas y cumplir estándares de calidad.	Mantenibilidad	Capacidad de prueba, Analizabilidad	M_M-01, M_M-02	pytest-cov + Jest, ESLint/PyLint	CP-M-01: Ejecución de suite de pruebas unitarias y reporte de cobertura
REQ-05	El sistema debe emitir información adecuada (mensajes de estado, error y éxito) que guíe al usuario durante la interacción.	Adecuación Funcional	Apropiación funcional	M_CI-01 (ítems 6-7)	Playwright + axe-core	CP-CI-02: Verificación de mensajes de error claros y accionables
REQ-06	El código fuente debe cumplir con estándares de nomenclatura (inglés), formato y ausencia de código duplicado o sin usar.	Mantenibilidad	Analizabilidad	M_M-02	ESLint + PyLint + Prettier + vulture	CP-M-02: Análisis estático de código y reporte de cumplimiento o de estándares

3.5 Criterios de Inicio, Suspensión y Reanudación de la Evaluación

Para gestionar eficientemente los recursos y asegurar la calidad de los resultados, se definen las siguientes condiciones operativas para la ejecución de las pruebas:

Criterios de Inicio:

- El entorno de desarrollo (Docker Compose) debe estar levantado y estable en el servidor de pruebas local.
- Los modelos Pydantic para los 14 artefactos deben estar definidos y versionados en el repositorio.
- La suite básica de pruebas unitarias debe compilar sin errores de sintaxis.
- Los archivos de configuración de análisis estático (pylintrc, .eslintc, .prettierrc) deben estar definidos y versionados en el repositorio.
- Los datos de prueba (fixtures) deben estar cargados en la base de datos de testing.

Criterios de Suspensión:

- Si más del 5% de las pruebas de integración fallan debido a errores de infraestructura (ej. caída de contenedores Docker, problemas de conexión a BD), se suspenderá la ejecución para priorizar la estabilización del entorno.
- Si se detecta un bloqueo crítico en el pipeline de LangChain que impide la generación de cualquier artefacto (error 500 generalizado).
- Si las herramientas de medición (pytest, Jest, axe-core, Lighthouse CI) presentan inconsistencias técnicas que invalidan los reportes de cobertura o accesibilidad.

Criterios de Reanudación:

- Una vez corregidos los errores de infraestructura que causaron la suspensión y validados mediante una ejecución preliminar exitosa ("smoke test").
- Tras la implementación de parches correctivos para los bloqueos del pipeline, confirmados por el equipo de desarrollo.

Criterios de Aprobación Final del Plan:

El plan de pruebas se considerará cumplido y el producto apto para la sustentación académica si:

1. Las 5 métricas principales cumplen o superan sus criterios de aceptación:
 - M_AF-01 (Artefactos con Estructura Válida): $\geq 85\%$
 - M_AF-02 (Tasa de Éxito en Integración): $\geq 85\%$
 - M_CI-01 (Accesibilidad y Consistencia): $\geq 80\%$
 - M_M-01 (Cobertura de Pruebas Unitarias): $\geq 80\%$
 - M_M-02 (Estándares de Código): $\geq 85\%$
2. No existen incidencias críticas abiertas relacionadas con la Adecuación Funcional.
3. El informe de resultados ha sido revisado y validado por el director del proyecto de grado.

SECCIÓN 4: INSTRUMENTOS DE EVALUACIÓN

Esta sección detalla las herramientas específicas, los procedimientos de diseño y los métodos de recolección de datos seleccionados para operacionalizar las métricas definidas en la Sección 3. La selección de instrumentos responde estrictamente a las restricciones del

proyecto: viabilidad académica, ausencia de grandes volúmenes de datos y enfoque en la integración técnica (MVP). Cada instrumento está alineado con el stack tecnológico validado (Next.js 15, FastAPI, LangChain, PostgreSQL, Docker) y los atributos de calidad ISO/IEC 25010 seleccionados.

4.1 Herramientas Seleccionadas

La siguiente tabla presenta el conjunto de herramientas software que constituyen los instrumentos técnicos para la medición de las métricas. Se ha priorizado el uso de tecnologías nativas del ecosistema del proyecto para minimizar la curva de aprendizaje y maximizar la integración en el flujo de desarrollo continuo.

Atributo ISO 25010	Métrica Asociada	ID Métrica	Herramienta Principal	Justificación de la Selección
Adecuación Funcional	Porcentaje de Artefactos con Estructura Válida	M_AF-01	Pydantic (Python) + pytest	Pydantic permite definir esquemas de datos rigurosos para los 14 artefactos. Su integración nativa con FastAPI y pytest facilita la validación automática de cada respuesta del pipeline RAG sin necesidad de herramientas externas complejas.

Atributo ISO 25010	Métrica Asociada	ID Métrica	Herramienta Principal	Justificación de la Selección
Adecuación Funcional	Tasa de Éxito en Flujos de Integración	M_AF-02	Playwright (TypeScript) + httpx	Playwright se selecciona sobre Selenium por su mayor velocidad, soporte nativo para aplicaciones modernas basadas en React/Next.js y capacidad de manejar asincronía. httpx (TestClient) permite probar endpoints de FastAPI aisladamente sin levantar el servidor completo.
Capacidad de Interacción	Índice de Accesibilidad y Consistencia de Interfaz	M_CI-01	axe-core + Lighthouse CI + Playwright	Herramientas automatizadas que validan accesibilidad WCAG 2.1 AA, consistencia visual y navegación sin requerir usuarios reales. Se integran nativamente con el pipeline de pruebas E2E ya configurado con Playwright.

Atributo ISO 25010	Métrica Asociada	ID Métrica	Herramienta Principal	Justificación de la Selección
Mantenibilidad	Cobertura de Pruebas Unitarias	M_M-01	pytest-cov (Backend) + Jest (Frontend)	Son los estándares de la industria para Python y JavaScript/Type Script respectivamente. Generan reportes detallados (HTML/XML) integrables en pipelines de CI/CD locales sin configuración adicional costosa.
Mantenibilidad	Cumplimiento de Estándares de Código	M_M-02	ESLint + Pylint + Prettier + vulture	Herramientas de análisis estático que validan nomenclatura en inglés, convenciones de formato, ausencia de código duplicado y código sin usar. Obligatorias según los comentarios del director de evaluación.

SECCIÓN 5: PLANIFICACIÓN Y EJECUCIÓN DE LA EVALUACIÓN

Esta sección operacionaliza el plan de evaluación definido en las secciones anteriores, estableciendo el cronograma, los responsables, los recursos y la estrategia específica para la recolección de datos. En concordancia con la naturaleza iterativa del desarrollo de software bajo metodologías ágiles y las directrices del Taller de Planificación de Evaluación del PDG de la Universidad ICESI, la ejecución de las pruebas no se concibe como una fase final aislada, sino como una actividad continua integrada en cada ciclo de desarrollo de nuevas funcionalidades.

5.1 Cronograma de Recolección de Datos (Enfoque Iterativo)

La recolección de datos se alinearán con los hitos de desarrollo del proyecto. A medida que se implemente cada módulo funcional (AS-IS, TO-BE, Brechas, Roadmap), se ejecutarán las pruebas correspondientes a ese incremento. Esto permite detectar desviaciones de calidad tempranamente y corregirlas antes de avanzar a la siguiente iteración.

Fase / Iteración	Funcionalidad Objetivo	Actividades de Evaluación	Métricas a Recolectar	Entregable de Evidencia
Iteración 1: Configuración y Cimientos	Arquitectura base, Docker, Esquemas Pydantic iniciales.	- Configuración de entornos de prueba (pytest, Jest). - Definición de modelos Pydantic para artefactos AS-IS. - Configuración de análisis estático (ESLint, Pylint). - Pruebas unitarias de servicios básicos.	- M_M-01 (Cobertura inicial). - M_M-02 (Estándares de código inicial). - M_AF-01 (Validación de esquemas base).	- Reporte de cobertura inicial. - Reporte de estándares de código. - Archivos de modelos Pydantic versionados.

Fase / Iteración	Funcionalidad Objetivo	Actividades de Evaluación	Métricas a Recolectar	Entregable de Evidencia
Iteración 2: Módulo AS-IS	Generación de Diagrama ER, Inventario, RACI, DFD,	- Pruebas de integración API (FastAPI + LangChain). - Validación estructural de los 4 artefactos AS-IS. - Pruebas E2E del flujo completo AS-IS.	- M_AF-01 (% Artefactos válidos AS-IS). - M_AF-02 (Tasa de éxito flujo AS-IS).	- Logs de ejecución de pytest/Playwright. - Capturas de artefactos JSON válidos.
Iteración 3: Módulos TO-BE y Brechas	Generación de modelos lógicos, glosarios y reporte de brechas.	- Extensión de pruebas de integración a nuevos endpoints. - Validación estructural de artefactos TO-BE y Brechas. - Refinamiento de mocks de LangChain. - Ejecución de análisis estático de código.	- M_AF-01 (% Artefactos válidos acumulados). - M_M-01 (Cobertura actualizada). - M_M-02 (Estándares de código actualizados).	- Reporte de cobertura actualizado (≥80%). - Reporte de cumplimiento de estándares (≥95%). - Suite de pruebas de integración ampliada.
Iteración 4: Módulo Roadmap y UI Final	Gantt, KPIs, Estándares y consolidación de interfaz.	- Pruebas E2E de todos los flujos críticos. - Ejecución de pruebas de accesibilidad automatizadas (axe-core + Lighthouse CI). - Ajuste final de cobertura de pruebas.	- M_CI-01 (Accesibilidad y Consistencia). - M_AF-02 (Tasa de éxito global). - M_M-01 (Cobertura final).	- Reportes automatizados de accesibilidad WCAG 2.1 AA. - Reportes de Lighthouse CI. - Logs de ejecución de Playwright.

Fase / Iteración	Funcionalidad Objetivo	Actividades de Evaluación	Métricas a Recolectar	Entregable de Evidencia
Iteración 5: Cierre y Consolidación	Estabilización general y preparación para sustentación.	- Ejecución completa de la suite de pruebas (Regression Testing). - Verificación de cumplimiento de todos los criterios de aceptación. - Generación de reportes finales.	- Todas las métricas (Validación final > umbrales): - M_AF-01 $\geq$ 85% - M_AF-02 $\geq$ 85% - M_CI-01 $\geq$ 80% - M_M-01 $\geq$ 80% - M_M-02 $\geq$ 85%	- Informe Final de Evaluación. - Evidencias compiladas para anexos de tesis. - Reportes consolidados de las 5 métricas.

5.2 Recursos Requeridos para Implementar el SubProceso de Pruebas

Recursos Hardware:

- Portátil de desarrollo (Mínimo recomendado: 16 GB RAM, procesador multicore para contenedores Docker y modelos de IA locales si aplicara).
- Conexión a internet estable para acceso a APIs externas y repositorios de paquetes.

Recursos Software:

- Entorno de Desarrollo: Visual Studio Code o similar, Docker Desktop, Git.
- Stack de Pruebas: Python 3.10+, Node.js 18+, pytest, Jest, Playwright, Pydantic.
- Gestión de Datos: PostgreSQL (instancia Docker), clientes de base de datos (DBeaver/pgAdmin).
- Documentación: Procesador de texto (Google Docs/Word) para informes, hojas de cálculo para tabulación de usabilidad.

Recursos Humanos:

- Evaluador Principal: El estudiante investigador (ejecución técnica).
- Validador Académico: Director de tesis (aprobación de criterios y resultados).
- Usuarios de Prueba: 3 a 5 compañeros de programa o profesionales junior disponibles para sesiones de usabilidad (sin costo económico, bajo modalidad de apoyo académico entre pares).

5.3 Estrategia de Pruebas por Componente

La estrategia de evaluación se segmenta según la capa tecnológica para asegurar una cobertura adecuada de cada atributo de calidad, manteniendo la independencia entre componentes cuando sea posible.

Tabla 5.3.1: Resumen de Estrategia de Pruebas por Componente

Componente	Enfoque	Tactica Principal	Atributo ISO 25010 Asociado
Frontend (Next.js 15)	Validar la correcta representación de datos y la interacción del usuario	Pruebas Unitarias (Jest) + Pruebas E2E con mocks de backend (Playwright)	Capacidad de Interacción + Adecuación Funcional
Backend (FastAPI) + Orquestación (LangChain)	Validar la lógica de negocio, la integridad de datos y la correcta invocación de servicios de IA	Pruebas de Integración de API (httpx) + Pruebas de Pipeline RAG + Pruebas de Base de Datos	Adecuación Funcional + Mantenibilidad

5.4 Manejo de Riesgos en la Evaluación

Se han identificado posibles riesgos que podrían afectar la ejecución del plan de pruebas, junto con sus respectivas estrategias de mitigación:

Riesgo Identificado	Probabilidad	Impacto	Estrategia de Mitigación
Inestabilidad en la API externa (OpenAI)	Media	Alto	Uso sistemático de mocks de langchain.chat_models mediante unittest.mock en pruebas automatizadas. Las pruebas E2E reales se limitarán a una ventana de tiempo controlada con monitoreo de estado del servicio. La validación de estructuras de salida se realiza contra esquemas Pydantic, independientemente del contenido semántico generado por el LLM.
Configuración inconsistente de herramientas de análisis estático (ESLint/PyLint)	Media	Medio	Los archivos de configuración (.eslintrc, .pylintrc, .prettierrc) deben estar versionados en el repositorio y validados en la Iteración 1. Se ejecutará un script de verificación de configuración como parte del pipeline de CI/CD local antes de cada ejecución de M_M-02.
Inconsistencia entre pruebas E2E y de integración que afecta la medición de M_AF-02	Baja	Alto	Todas las pruebas E2E y de integración utilizarán los mismos fixtures estructurados y validarán contra los mismos esquemas Pydantic. Se mantendrá una matriz de trazabilidad actualizada que vincule cada caso de prueba con la métrica M_AF-02 para garantizar coherencia en la medición.
Complejidad en la definición de esquemas Pydantic para artefactos complejos	Alta	Medio	Enfoque iterativo: definir primero los esquemas de los artefactos más simples (ej. Inventario) y refinar progresivamente los complejos (ej. DFD) a medida que avanza el desarrollo. Consulta temprana con el director de tesis sobre la granularidad requerida para la validación estructural.

Riesgo Identificado	Probabilidad	Impacto	Estrategia de Mitigación
Deuda técnica que reduzca la cobertura de pruebas (M_M-01)	Media	Bajo	Integración de la revisión de cobertura como puerta de calidad (quality gate) antes de fusionar cualquier nueva funcionalidad. No se aceptará código nuevo sin sus pruebas unitarias correspondientes. Se establecerá un umbral mínimo del 80% en componentes críticos.